# Prof-K: Probabilistic One-Pass Filtering for Efficient Top-$k$ Selection

**Anonymous Manuscript**

## Abstract

Top-$k$ selection is a fundamental computational primitive with applications spanning databases, information retrieval, signal processing, and modern machine learning workloads, including sparse activations and attention pruning. As data sizes grow, existing approaches become inefficient: exact methods incur high memory and compute overhead, while approximate methods often rely on brittle heuristics that degrade under adversarial or heavy-tailed inputs. In this paper, we introduce Prof-K, a fast, scalable, and distribution-agnostic top-$k$ algorithm with probabilistic correctness guarantees. Prof-K performs a single-pass filtering procedure: a small random sample estimates an adaptive threshold, the $N$ input elements are streamed once into a compact buffer, and an exact top-$k$ routine on this buffer recovers the true top-$k$ elements with probability at least $1 - \varepsilon$, where $\varepsilon > 0$ is user specified. We derive high-probability guarantees for correctness and buffer size, together with an approximately optimal sample size that minimizes overhead as a function of $N$ and $k$. Empirically, Prof-K achieves $1.5\times$–$10\times$ speedups over the highly optimized PyTorch `topk` and recent RadiK implementations, with the largest gains in the large-scale, small-to-moderate-$k$ regime where prior methods struggle most. Unlike previous approaches, these guarantees hold independently of the input distribution, ensuring robustness to adversarial settings. By relaxing the recall target (e.g., recovering 95% of the true top-$k$ values), Prof-K additionally provides a principled accuracy–speed trade-off. We further demonstrate its impact on training BatchTopK Sparse Autoencoders (SAEs), where top-$k$ selection constitutes a significant portion of the training cost. Code accompanying this manuscript is available in the associated GitHub repository.

## 1 Introduction

Top-$k$ selection is a common systems operation that arises whenever only a small subset of the largest elements must be retained from a much larger collection [1, 2]. As tensor sizes continue to grow, the cost of repeated top-$k$ execution becomes increasingly significant in large-scale GPU workloads, particularly in sparse computation settings where only a small fraction of values is ultimately retained [3, 4, 5, 6].

The challenge is most visible in the large-$N$, small-$k$ regime. Even when only a tiny fraction of elements is needed, exact algorithms must still determine the precise global decision boundary across the full tensor. On modern GPUs, this cost is dominated primarily by memory movement and synchronization, making top-$k$ disproportionately expensive relative to the surrounding computation [7, 8, 9, 10, 11].

In this work, we introduce Prof-K, a probabilistic one-pass filtering algorithm for efficient top-$k$ selection. The method first draws a small uniform sample to estimate a conservative threshold, then streams once through the input tensor while retaining only elements above this threshold in a compact candidate buffer. Exact top-$k$ refinement is finally applied only to the retained candidates. Because the analysis depends only on ranks induced by uniform sampling, the resulting guarantees

Preprint.

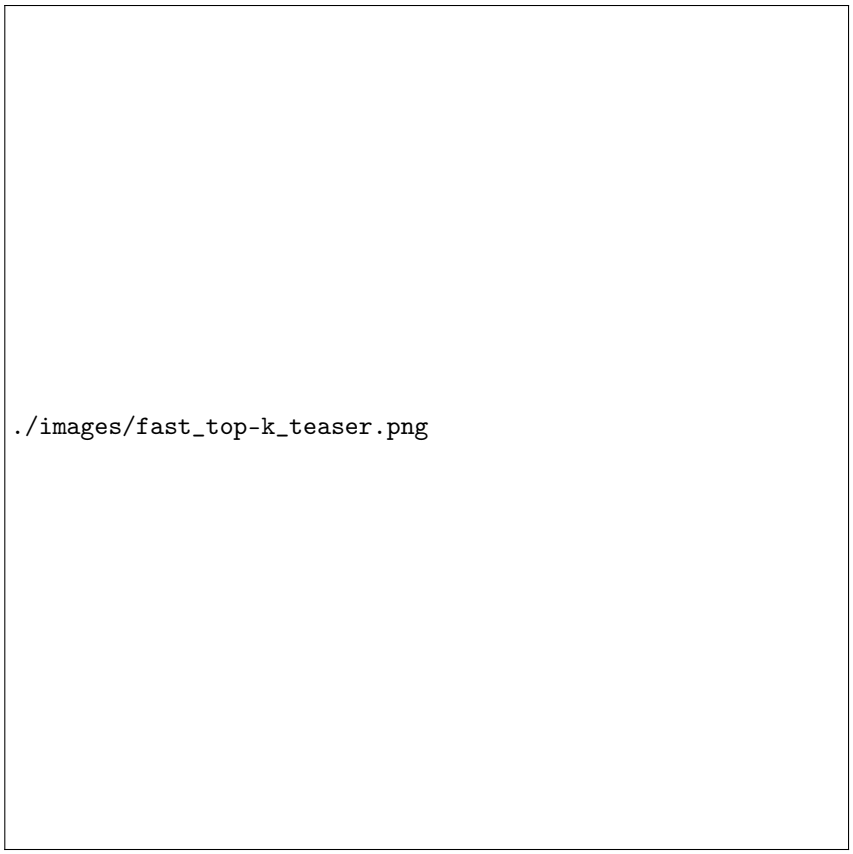


Figure 1: **The core idea of our Prof-K algorithm.** *Top:* Full data of size $N$ sorted in descending order; the true top-$k$ boundary is marked by a vertical dashed line. *Middle:* A random sample (solid outlines) is used to compute a threshold $\tau$ (green dashed line) as a conservative estimate of the $(k/N)$-quantile. *Bottom:* The threshold filter is applied in parallel to the full data. Elements $\geq \tau$ (green) are retained in a compact candidate buffer, while elements below $\tau$ are discarded. This approach ensures all true top-$k$ elements are preserved with high probability while drastically reducing the workload for the final selection stage.

are distribution-agnostic and remain valid under heavy-tailed or adversarial inputs. A schematic summary of Prof-K is presented in Figure 1.

We derive explicit parameter choices controlling both recall failure and buffer overflow, together with an approximately optimal sample size scaling as $(kN)^{1/3}$ in the sparse regime. Empirically, Prof-K achieves substantial speedups over optimized baselines such as PyTorch `topk` and RadiK [11], particularly for very large tensors and small retained fractions.

We additionally evaluate Prof-K in BatchTopK Sparse Autoencoder (SAE) [6] training. Even on a relatively small language model, we observe measurable reductions in end-to-end training time without changes in reconstruction quality or sparsity behavior. Since the flattened BatchTopK selection problem scales directly with model and dictionary size, we expect the advantages of Prof-K to become even stronger in larger SAE settings such as GPT-2 Small [12] and Gemma 2 2B [13].

Overall, Prof-K provides three main benefits:

- **Efficiency in the large-$N$, small-to-moderate-$k$ regime.** Prof-K reduces exact selection over $N$ elements to exact refinement on a small compact candidate buffer, with benefits that grow as $N$ increases.
- **Distribution-agnostic guarantees.** Unlike quantile-estimation methods that assume well-behaved distributions, Prof-K derives distribution-agnostic guarantees from random sampling and depends only on ranks rather than values, making it robust to heavy-tailed or adversarial inputs.

- **Flexible accuracy-speed tradeoffs.** Users can tune the sampling and buffer parameters of Prof-K to trade exact recovery for additional speed while retaining a fallback path to exact top-$k$ when needed.

## 2 Related Work

Top-$k$ selection is a fundamental primitive with applications in databases, information retrieval, signal processing, and modern machine learning systems. Classical exact methods are based on sorting, heaps, partition-based selection, or partial sorting routines such as quickselect and `nth_element`. Efficient top-$k$ query execution has long been studied in database systems and parallel hardware settings [1, 2]. In large-scale GPU workloads, however, the dominant cost is often memory movement rather than comparison complexity alone, making exact top-$k$ expensive even in optimized libraries such as PyTorch.

A substantial line of work studies efficient exact top-$k$ selection on GPUs. Early work by Alabi et al. [7] developed GPU-specific $k$-selection algorithms optimized for massively parallel architectures. Dr. Top-k [8] introduces a delegate-centric design that improves load balancing and reduces redundant work among GPU threads. Zhang et al. [9] provide a comprehensive study of parallel GPU top-$k$ algorithms and propose optimized implementations for different operating regimes. RTop-K [10] focuses on efficient row-wise top-$k$ selection and demonstrates strong performance for moderate-size neural network workloads. Radix-based approaches such as RadiK [11] further exploit floating-point representations to accelerate exact selection. In contrast, Prof-K reduces the size of the exact selection problem itself through probabilistic filtering, making it particularly effective for very large $N$ and relatively small $k$.

Approximate and probabilistic selection methods reduce computational cost using sampling, sketching, or approximate quantile estimation, often at the cost of weaker guarantees or assumptions on the input distribution. Quantile summary methods such as the Greenwald–Khanna [14] algorithm provide efficient streaming approximations with bounded error guarantees. Prof-K differs in that its guarantees depend only on the combinatorial properties of uniform sampling without replacement. The analysis therefore depends on ranks rather than values, yielding distribution-agnostic probabilistic guarantees.

Top-$k$ selection is increasingly important in sparse machine learning workloads, including mixture-of-experts routing, sparse attention, activation pruning, and sparse autoencoders [3, 4, 5, 6]. These workloads commonly operate in the regime targeted by Prof-K: extremely large tensors with small retained fractions. Rather than replacing exact selection entirely, Prof-K uses sampling to construct a conservative threshold and then applies exact top-$k$ refinement on a compact candidate buffer, making it complementary to optimized GPU kernels such as Dr. Top-k, RTop-K, and RadiK.

## 3 Problem Setup and Algorithm Overview

Given an input vector $x \in \mathbb{R}^N$ and an integer $k \geq 1$, our goal is to return the $k$ largest entries of $x$ together with their indices. Let $\alpha = k/N$ denote the target fraction of retained elements. For batched inputs with batch size $B$, the procedure is applied in parallel to each batch element.

**The Prof-K Algorithm** Prof-K proceeds in four stages, designed to minimize memory traffic while providing probabilistic correctness guarantees:

1. **Sampling.** Uniformly sample $S$ indices without replacement from $[N] = \{1, \ldots, N\}$. Let $\mathcal{S} \subset [N]$ denote the sampled indices.
2. **Threshold Estimation.** Sort the sampled values $x|_{\mathcal{S}}$ in descending order. For a chosen rank $t \in [S]$, define the threshold $\tau = s_t$, where $s_t$ is the $t$-th largest sampled value.
3. **Filtering.** Stream through the full input once, appending every element satisfying $x_i \geq \tau$ to a candidate buffer $\mathcal{B}$ of capacity $M$, which is a small constant multiple of $k$.
4. **Refinement.** Run an exact top-$k$ routine on the buffer $\mathcal{B}$ to recover the final answer. If fewer than $k$ candidates are collected, or if the buffer overflows before the scan completes, Prof-K falls back to an exact top-$k$ computation on the full input.

The central design goal is to choose the parameters $(S, t, M)$ so that the number of retained elements $R = |\{i \in [N] : x_i \geq \tau\}|$ satisfies $k \leq R \leq M$ with probability at least $1 - \varepsilon$, where $0 < \varepsilon \ll 1$ is user specified. When this holds, the final exact top-$k$ operates on only a small multiple of $k$ elements rather than all $N$ entries.

**Key Insight: Why Sampling Works** The core idea is that a random sample provides an unbiased estimate of quantile positions. If we select the $(t/S)$-quantile from the sample, it approximates the $(t/S)$-quantile of the full population. By choosing $t$ slightly larger than $S\alpha = Sk/N$, we obtain a threshold that is conservatively low, ensuring all true top-$k$ elements pass the filter with high probability, while still excluding most of the remaining elements.

## 4 Theoretical Analysis

We now formalize the probabilistic guarantees underlying the Prof-K algorithm. Our analysis proceeds in three steps: (1) characterizing the distribution of the estimated threshold rank, (1) deriving parameter choices that bound the failure probability, and (3) optimizing the sample size to minimize computational overhead. The omitted proofs are provided in Appendix C.

**Distribution of the Threshold Rank** The first key observation is that the rank of our sample-based threshold in the full population follows a well-understood distribution, regardless of the input values themselves.

**Lemma 1** (Threshold rank distribution)**.** *Let $\mathcal{S} \subset [N]$ be a uniform sample of size $S$ drawn without replacement. Let $\tau$ be the $t$-th largest sampled value, and let $R$ denote its rank in the fully sorted input (rank 1 is the largest). Then $R$ follows the negative hypergemetric distribution, i.e., for $r \in [t, N - S + t]$,*

$$\Pr(R = r) = \frac{\binom{r-1}{t-1}\binom{N-r}{S-t}}{\binom{N}{S}}. \tag{1}$$

*Moreover,*

$$\mathbb{E}[R] = \frac{N+1}{S+1}\,t, \quad \mathrm{Var}(R) = \frac{t(S-t+1)(N+1)(N-S)}{(S+1)^2(S+2)}. \tag{2}$$

This result is powerful because *it depends only on the ranks, not on the actual values in $x$*. Whether the input is uniformly distributed, heavy-tailed, or adversarially constructed, the same probabilistic bounds apply. For practical parameter regimes, a Gaussian approximation provides convenient closed-form expressions.

**Lemma 2** (Normal approximation)**.** *Suppose $N, S \to \infty$ with $S/N \to f_\infty \in (0, 1)$. Let $q = t/S$. Then*

$$\frac{R - Nq}{N\sqrt{\frac{q(1-q)}{S}\left(1 - \frac{S}{N}\right)}} \xrightarrow{d} \mathcal{N}(0, 1). \tag{3}$$

*Hence, for large finite $N, S$,*

$$R \approx \mathcal{N}\left(Nq,\ N^2\frac{q(1-q)}{S}f\right), \quad f = 1 - \frac{S}{N}. \tag{4}$$

The factor $f = 1 - S/N$ is the finite-population correction: sampling without replacement reduces variance when the sample constitutes a non-negligible fraction of the population.

**Failure Modes and Budget Allocation** Prof-K can fail in two distinct ways, each requiring separate treatment:

1. **Insufficient Recall.** The threshold is too high ($R < k$), so fewer than $k$ valid candidates survive filtering.
2. **Buffer Overflow.** Too many elements exceed the threshold ($R > M$), and some true top-$k$ entries fail to enter the finite buffer.

We allocate a total failure budget $\varepsilon = \varepsilon_A + \varepsilon_B$ across these events, with $\varepsilon_A$ controlling recall failures and $\varepsilon_B$ controlling overflow.

**Controlling Insufficient Recall** This failure mode corresponds to the event $\{R < k\}$. Using the Gaussian approximation in Equation (4), we obtain

$$\Pr(R < k) \approx \Phi\left(\frac{k - Nq}{N\sqrt{\frac{q(1-q)}{S}f}}\right) \leq \varepsilon_A, \tag{5}$$

where $\Phi$ is the standard normal CDF. We express the sampling quantile rank as $q = \alpha + \delta$ with $\alpha = k/N$, and note that for small $\delta$ we have $k - Nq = -N\delta$ and $q(1-q) \approx \alpha(1-\alpha)$. Substituting into the tail bound yields $\Phi\left(-\delta/\sqrt{\frac{\alpha(1-\alpha)}{S}f}\right) \leq \varepsilon_A$, which implies $\delta \geq z_A\sqrt{\frac{\alpha(1-\alpha)f}{S}}$ with $z_A = \Phi^{-1}(1-\varepsilon_A)$. This leads to the threshold choice

$$q = \alpha + z_A\sqrt{\frac{\alpha(1-\alpha)f}{S}}, \qquad t = \left\lceil S\alpha + z_A\sqrt{S\alpha(1-\alpha)f}\,\right\rceil, \tag{6}$$

which can be interpreted as shifting the expected sample rank $S\alpha$ upward by $z_A$ standard deviations of its sampling distribution to reduce the probability of discarding true top-$k$ elements; when $S \ll N$, this simplifies to the standard binomial form $t \approx \alpha S + z_A\sqrt{\alpha(1-\alpha)S}$. In the extreme sparse regime $\alpha S \ll 1$, the Gaussian approximation is unreliable, and a conservative alternative is to set $t = 1$, yielding failure probability $\Pr(\mathrm{Bin}(S,\alpha) = 0) = (1-\alpha)^S \approx e^{-\alpha S}$, so that $\alpha S \geq -\ln\varepsilon_A$ ensures the desired bound; otherwise (rarely), the algorithm falls back to exact top-$k$ computation.

**Controlling Buffer Overflow** This failure mode is (more than) covered by the event $\{R > M\}$. Let $M = ck$ with buffer multiplier $c \geq 1$. Using the threshold choice from Equation (6), the expected number of retained elements is $\mathbb{E}[R] = Nq$, which can be written as

$$\mathbb{E}[R] = k + Nz_A\sqrt{\frac{\alpha(1-\alpha)f}{S}} = k\left(1 + \frac{z_A}{\sqrt{S}}\sqrt{\frac{1-\alpha}{\alpha}f}\right), \tag{7}$$

motivating the definition of the nominal buffer multiplier

$$c_0 = 1 + \frac{z_A}{\sqrt{S}}\sqrt{\frac{1-\alpha}{\alpha}f}. \tag{8}$$

The variability of the retained set is governed by

$$\mathrm{Var}(R) = \sigma_R = N\sqrt{\frac{q(1-q)}{S}f} \approx k\sqrt{\frac{1-\alpha}{\alpha S}f}, \tag{9}$$

so that by a Gaussian approximation the overflow probability satisfies

$$\Pr(R > ck) \approx \Phi\left(-\frac{(c-c_0)k}{\sigma_R}\right) = \Phi\left(-(c-c_0)\sqrt{\frac{\alpha S}{(1-\alpha)f}}\right). \tag{10}$$

Enforcing this to be at most $\varepsilon_B$ yields

$$c = c_0 + z_B\sqrt{\frac{(1-\alpha)f}{\alpha S}}, \qquad z_B = \Phi^{-1}(1-\varepsilon_B). \tag{11}$$

**Main Correctness Guarantee** Our main theoretical result follows immediately from the preceding analysis.

**Theorem 3** (Total failure probability bound)**.** *Assume a target failure budget $\varepsilon = \varepsilon_A + \varepsilon_B$ with $\varepsilon_A, \varepsilon_B \ll 1$ (typically $\varepsilon_A = \varepsilon_B = \varepsilon/2$). Choose $t$ according to Equation* (6)*, and set $M = \lceil ck \rceil$, where $c$ is given by Equation* (11)*. Then the total failure probability of Prof-K is at most $\varepsilon$.*

This result formalizes a practical design principle: one part of the failure budget controls threshold quality (ensuring we do not miss true top-$k$ elements), and the other controls buffer adequacy (ensuring we have room for all candidates).

**Algorithm 1** Adaptive parameter selection for Prof-K

**Require:** $N$ (tensor size), $k$ (top-$k$), $\varepsilon$ (target failure budget, default $10^{-3}$), $S_{\min}, S_{\max}$ (sample size bounds), $\tau_B/\tau_A$ (cost ratio, default 1)
**Ensure:** $S, t, M$ (sample size, threshold rank, buffer capacity)
1: $z \leftarrow \Phi^{-1}(1-\varepsilon/2) + \Phi^{-1}(1-\varepsilon/2)$ $\{\approx 6.58$ for $\varepsilon = 10^{-3}\}$
2: $\alpha \leftarrow k/N$
3: $S^* \leftarrow \left(\frac{z(\tau_B/\tau_A)\log_2 k}{2}\right)^{2/3} \cdot (k(N-k))^{1/3}$
4: $S \leftarrow \min(S_{\max}, \max(S_{\min}, \lfloor S^* \rceil))$
5: $f \leftarrow 1 - S/N$
6: $t \leftarrow \max\big(1,\ \lceil S\alpha + \frac{z}{2}\sqrt{S\alpha(1-\alpha)f}\,\rceil\big)$
7: $M \leftarrow \lceil k + z\sqrt{\frac{k(N-k)}{S}f}\,\rceil$
8: **if** $\alpha S < -\ln(\varepsilon/2)$ **then** $t \leftarrow 1$ `% fallback-prone regime`
9: **return** $S, t, M$

**Deriving the Buffer Size** Combining the expressions for $t$ and $c$ yields a compact closed-form for the buffer size.

**Corollary 4** (Closed-form buffer size)**.** *Let $z = z_A + z_B$. Then*

$$M = k + z\sqrt{\frac{k(N-k)}{S}\left(1 - \frac{S}{N}\right)}. \tag{12}$$

This expression shows that the excess buffer size $M - k$ decreases at the rate $O(1/\sqrt{S})$: larger samples produce more accurate thresholds and therefore require smaller buffers.

**Optimal Sample Size** Beyond the mandatory linear scan through the input, Prof-K incurs two additional costs: (i) sampling and selecting the threshold from $S$ values, and (ii) exact top-$k$ on a buffer of size $M$. We model total overhead as:

$$C = \tau_A S + \tau_B M \log k, \tag{13}$$

where $\tau_A$ and $\tau_B$ are hardware-dependent constants capturing the per-element cost of sample processing and of retaining the top-$k$ elements in the candidate buffer, respectively.

**Theorem 5** (Optimal sample size)**.** *When $S \ll N$ (so $f \approx 1$), the minimizer of $C$ satisfies*

$$S^* = \left(\frac{z\tau_B \log k}{2\tau_A}\right)^{2/3} \big(k(N-k)\big)^{1/3}. \tag{14}$$

*For the common regime $k \ll N$, this simplifies to $S^* \propto (kN)^{1/3}$.*

This scaling is noteworthy: the optimal sample size grows as $(kN)^{1/3}$, much more slowly than either $k$ or $N$ individually. For example, with $N = 10^9$ and $k = 100$, we have $S^* \approx 4{,}600$—a tiny fraction of the input.

In practice, we restrict $S$ to the interval $[S_{\min}, S_{\max}]$, where $S_{\min} = 32768$ avoids high-variance threshold estimates and $S_{\max}$ is chosen so that sampling overhead remains negligible relative to the mandatory $O(N)$ streaming pass (e.g., $S_{\max} = 2^{17} = 131{,}072$ in typical GPU implementations). The ratio $\tau_B/\tau_A$ may be estimated by lightweight microbenchmarking; a default choice of $\tau_B/\tau_A = 1$ performs robustly across a broad range of $(N, k)$. When $\alpha S < -\ln \varepsilon_A$ (very small $k$ combined with modest sample size), the Gaussian approximation becomes unreliable. In this regime, we conservatively set $t = 1$ (using the sample maximum as threshold), accepting occasional fallback to exact top-$k$ computation.

**Parameter Selection** Algorithm 1 summarizes the adaptive parameter selection for Prof-K. For additional discussion of the algorithmic details, see Section D.

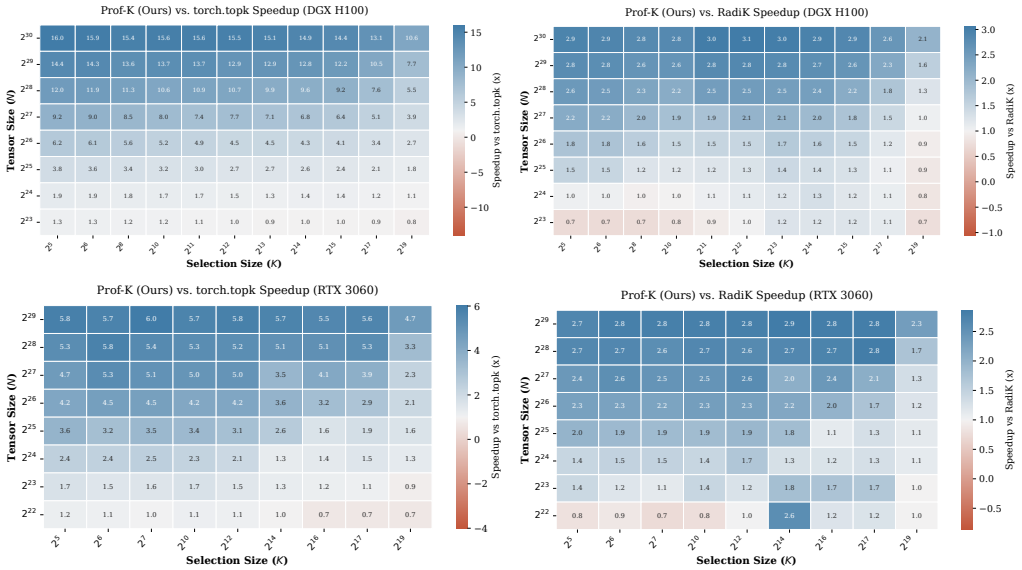


Figure 2: Average latency speedup of our method across the $(N, k)$ space. *Top:* Speedups on high-end GPU (DGX H100) vs `torch.topk` (left), and RadiK (right). *Bottom:* Speedups on consumer tier GPU (RTX 3060) vs `torch.topk` (left), and RadiK (right). Our approach exhibits substantial advantages in the large-$N$ regime.

## 5 Experiments

We evaluate Prof-K on synthetic benchmarks spanning a wide range of input sizes and sparsity levels, comparing against PyTorch `topk` and RadiK. We report wall-clock speedups over key hyper-parameters. Finally, we demonstrate practical impact by integrating Prof-K into BatchTopK Sparse Autoencoder (SAE) training, where top-$k$ selection is invoked at every optimization step over large flattened activation tensors. This setting tests whether kernel-level improvements translate to end-to-end training acceleration, and whether the probabilistic relaxation affects learned representations or reconstruction quality. Code accompanying this manuscript is available in the associated GitHub repository.

**Experimental Setup** We evaluate our Triton-based implementation on high-end data center (DGX H100) and consumer-grade (RTX 3060) GPUs, benchmarking against native `torch.topk` and RadiK [11]. Our synthetic workloads encompass tensor sizes $N \in [2^{22}, 2^{30}]$ and selection sizes $K \in [2^5, 2^{19}]$. We generate tensors using Uniform, Standard Normal, and heavy-tailed Pareto distributions. Crucially, because our algorithm uniformly samples indices rather than values, it is inherently distribution-agnostic; performance remains strictly invariant even under extreme outliers. For all exact top-$k$ evaluations, hyperparameters are conservatively configured to guarantee 100% accuracy.

**Algorithm Performance** Figure 2 illustrates global latency speedups across the $(N, k)$ parameter space. Our approach exhibits pronounced advantages in the memory-bandwidth-bound large-$N$ regime across both hardware tiers. By isolating tensor size (Figure 3), we observe that our filtering mechanism drastically flattens the latency scaling curve compared to the strict bandwidth limits governing baseline methods. Finally, strong scaling tests at constant size $N = 2^{29}$ (Figure 4) confirm that our method maintains robust speedups even as the absolute retrieval volume grows proportionally with the dataset. Latency remains highly competitive for large $k$.

**Scalability with Selection Size ($k$) and Memory Footprint.** A key advantage of our algorithm is its memory efficiency. Figure 5 fixes a massive tensor size (e.g., $N = 2^{29}$ for H100) and varies $k$. The paired bar chart demonstrates that our auxiliary memory consumption is negligible compared to SOTA radix-based methods which require $O(N)$ additional space on the GPU. This renders our approach impervious to Out-Of-Memory errors when the size $N$ becomes extremely large.

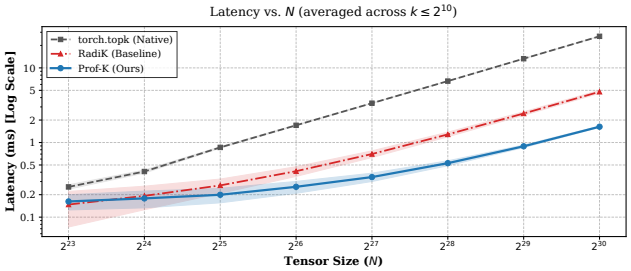


Figure 3: End-to-end latency (ms, log scale) as a function of $N$, averaged across $k$. Shaded regions denote standard deviations across different random input distributions. Prof-K achieves the best performance starting from $N = 2^{24}$.

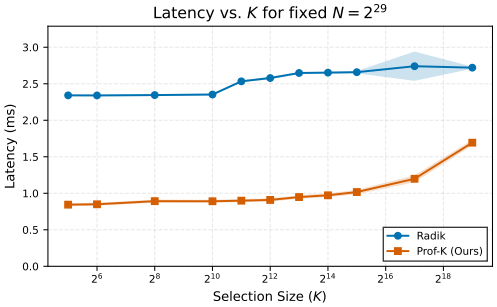


Figure 4: Latency plotted against $k$ for a large tensor size $N = 2^{29}$ on DGX H100. Both top-$k$ methods scale well with $k$.

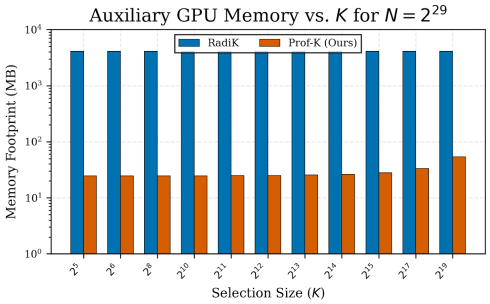


Figure 5: Auxiliary GPU memory overhead as a function of $k$ for a fixed tensor size ($N = 2^{29}$) (logarithmic scale). Our method's memory footprint is bounded by a function of the buffer capacity, avoiding OOM errors on massive arrays.

**Application: BatchTopK Sparse Autoencoders** To evaluate the practical impact of Prof-K in a real machine learning workload, we replace the exact batch-global top-$k$ operator inside BatchTopK Sparse Autoencoders (SAEs) with our probabilistic filtering procedure. BatchTopK SAEs are a natural testbed because top-$k$ selection is executed at every optimization step over a large flattened activation tensor, making it a recurring systems bottleneck during training. This setting therefore tests whether faster top-$k$ selection yields measurable end-to-end acceleration in a realistic sparse training pipeline rather than only in standalone synthetic benchmarks.

We consider the BatchTopK setup of Bussmann et al. [6], but focus on the computational cost of the selection primitive rather than introducing a new SAE objective. In each training step, the post-ReLU latent activations form a matrix of shape $B \times d_{\text{dict}}$, which is flattened into a vector of length $N = B \cdot d_{\text{dict}}$. BatchTopK then selects the largest $K = B \cdot k$ activations globally across the batch. This matches the operating regime targeted by Prof-K: very large $N$, relatively small retained fraction $K/N$, and repeated invocation inside the training loop.

In our experiment, we use activations from `EleutherAI/pythia-70m-deduped` at the MLP submodule of layer 1 and train BatchTopK SAEs on OpenWebText, following the same training hyperparameter regime as Bussmann et al. [6], namely batch size $B = 4096$, learning rate $3 \times 10^{-4}$, and target sparsity level $k = 32$. We run both methods for 30,000 training steps. For the dictionary size, we use $d_{\text{dict}} = 12288$, which is one of the standard settings reported in their GPT-2 Small experiments. Thus,

$$k = 32, \qquad d_{\text{dict}} = 12288, \qquad B = 4096,$$

which gives

$$N = B \cdot d_{\text{dict}} = 50{,}331{,}648, \qquad K = B \cdot k = 131{,}072.$$

The experiment is conducted on an NVIDIA DGX A100 system.

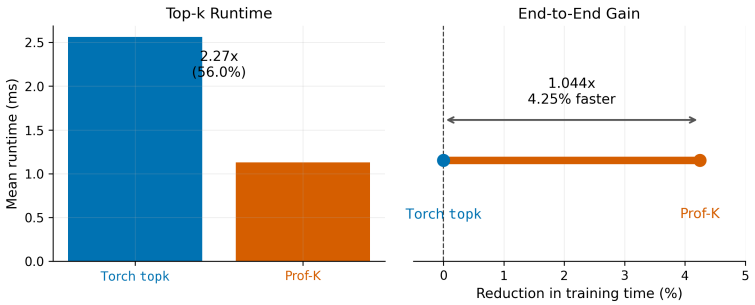


Figure 6: Runtime comparison in BatchTopK SAE training. *Left:* mean top-$k$ kernel runtime for Torch `topk` and Prof-K. *Right:* reduction in total training time induced by Prof-K, reported relative to Torch `topk`. Prof-K substantially reduces the cost of the top-$k$ primitive and yields a smaller but consistent improvement in overall wall-clock training time.

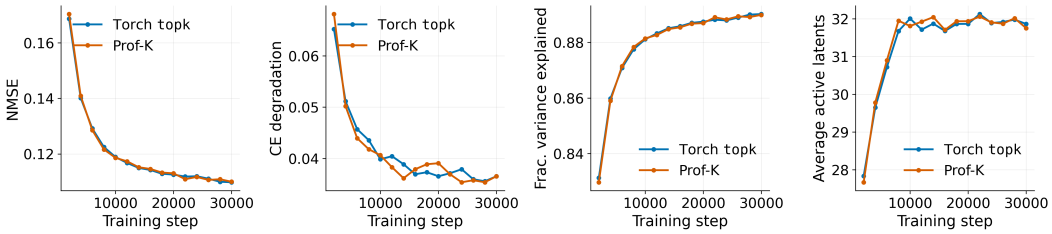


Figure 7: BatchTopK SAE training quality comparison. We compare Torch `topk` and Prof-K in terms of NMSE and cross-entropy degradation [6], as well as fraction of variance explained and average number of active latents per sample [15]. The curves remain closely matched throughout training, indicating that replacing exact top-$k$ with Prof-K preserves reconstruction quality, downstream behavior, and effective sparsity.

We compare a baseline implementation based on Torch `topk` against a Prof-K variant that uses probabilistic filtering followed by exact refinement on the retained candidate buffer. Both runs use the same optimizer, model activations, and SAE hyperparameters, differing only in the implementation of the batch-global top-$k$ routine.

The systems results are shown in Figure 6. Prof-K reduces mean top-$k$ time from approximately $2.56$ ms to $1.13$ ms, corresponding to a $2.27\times$ kernel-level speedup. At the full training-step level, the effect is smaller but still clearly measurable: mean step time decreases from approximately $30.73$ ms to $29.42$ ms, corresponding to a $1.044\times$ end-to-end speedup. This translates into a total training-time reduction of approximately $4.25\%$.

The quality results are shown in Figure 7. We track normalized mean squared error (NMSE), cross-entropy degradation, fraction of variance explained, and average active latents per sample. Across these metrics, the Prof-K and Torch `topk` curves are nearly indistinguishable: reconstruction quality improves at the same rate, sparsity remains matched to the intended operating point, and downstream degradation remains comparable throughout training. This indicates that replacing exact selection with Prof-K preserves the optimization behavior of the surrounding SAE training pipeline.

Taken together, these results show that Prof-K is useful beyond top-$k$ microbenchmarks. In BatchTopK SAE training, it reduces the cost of the top-$k$ primitive while preserving training dynamics and final reconstruction quality. Although the resulting end-to-end speedup in this setting is modest, even a few percent reduction in per-step runtime can translate into hours of wall-clock savings for larger language models or longer-running training workloads.

## 6 Conclusions

This work proposes Prof-K, a probabilistic one-pass filtering method for efficient top-$k$ selection from $N$ input elements, achieving the largest gains in the large-scale-$N$, small-to-moderate-$k$ regime. By estimating a conservative threshold from a small random sample, Prof-K reduces the cost of exact selection while providing distribution-agnostic correctness guarantees. Our theoretical analysis establishes explicit bounds on recall failure and buffer overflow probabilities, together with an approximately optimal sample size scaling as $(kN)^{1/3}$. Experiments on synthetic benchmarks and BatchTopK SAE training demonstrate substantial reductions in top-$k$ latency while preserving downstream training quality and sparsity behavior. These results show that probabilistic filtering can serve as an effective and robust systems-level optimization for large-scale sparse machine learning workloads.

Limitations of the Prof-K algorithm are discussed in Appendix A.

## References


[1] Reza Akbarinia, Esther Pacitti, and Patrick Valduriez. Best position algorithms for efficient top-k query processing. *Information Systems*, 36(6):973–989, 2011.

[2] Anil Shanbhag, Holger Pirk, and Samuel Madden. Efficient top-k query processing on massively parallel hardware. In *Proceedings of the 2018 International Conference on Management of Data*, pages 1557–1570, 2018.

[3] N Shazeer, A Mirhoseini, K Maziarz, A Davis, Q Le, G Hinton, and J Dean. The sparsely-gated mixture-of-experts layer. *Outrageously large neural networks*, 2:2, 2017.

[4] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

[5] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[6] Bart Bussmann, Patrick Leask, and Neel Nanda. Batchtopk sparse autoencoders. *arXiv preprint arXiv:2412.06410*, 2024.

[7] Tolu Alabi, Jeffrey D Blanchard, Bradley Gordon, and Russel Steinbach. Fast k-selection algorithms for graphics processing units. *Journal of Experimental Algorithmics (JEA)*, 17:4–1, 2012.

[8] Anil Gaihre, Da Zheng, Scott Weitze, Lingda Li, Shuaiwen Leon Song, Caiwen Ding, Xiaoye S Li, and Hang Liu. Dr. top-k: delegate-centric top-k on gpus. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14, 2021.

[9] Jingrong Zhang, Akira Naruse, Xipeng Li, and Yong Wang. Parallel top-k algorithms on gpu: A comprehensive study and new methods. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–13, 2023.

[10] Xi Xie, Yuebo Luo, Hongwu Peng, and Caiwen Ding. Rtop-k: Ultra-fast row-wise top-k selection for neural network acceleration on gpus. In *The Thirteenth International Conference on Learning Representations*, 2024.

[11] Yifei Li, Bole Zhou, Jiejing Zhang, Xuechao Wei, Yinghan Li, and Yingda Chen. Radik: scalable and optimized gpu-parallel radix top-k selection. In *Proceedings of the 38th ACM International Conference on Supercomputing*, pages 537–548, 2024.

[12] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[13] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[14] Michael Greenwald and Sanjeev Khanna. Space-efficient online computation of quantile summaries. *ACM SIGMOD Record*, 30(2):58–66, 2001.

[15] Samuel Marks, Adam Karvonen, and Aaron Mueller. dictionary_learning. `https://github.com/saprmarks/dictionary_learning`, 2024.

[16] Herbert A David and Haikady N Nagaraja. *Order statistics*. John Wiley & Sons, 2004.

# A Limitations

The primary limitation of Prof-K is that its benefits become less pronounced for smaller input sizes $N$, where the overhead of sampling and filtering provides less advantage over highly optimized exact top-$k$ implementations. Additionally, the method relies on fallback to exact top-$k$ in rare cases where the probabilistic filter becomes unreliable or the candidate buffer overflows, introducing occasional overhead. Our experiments are currently limited to single-node GPU settings and sparse learning workloads based primarily on BatchTopK SAEs. Moreover, additional care is required when applying a batched version of Prof-K, as the probability of observing at least one failure increases with batch size. Extending the approach to distributed environments and broader machine learning workloads remains an important direction for future work.

# B Impact Statement and Declaration of LLM Usage

We do not anticipate direct negative societal impacts from this work. The proposed method is a systems-level optimization for efficient top-$k$ selection and does not introduce new capabilities beyond standard machine learning functionality. Any broader impact is indirect and arises from improved efficiency of existing models and workloads.

LLMs were used exclusively for language editing and text refinement to improve the clarity of the manuscript.

# C Proofs

**Proof of Theorem 1** The event $\{R = r\}$ requires: (i) exactly $t-1$ sampled indices among the top $r-1$ positions, (ii) the element at rank $r$ itself to be sampled, and (iii) the remaining $S-t$ sampled indices to come from ranks below $r$. Counting such configurations and dividing by $\binom{N}{S}$ yields Equation (1). Equation (2), which give the mean and variance, follows from standard properties of the negative hypergeometric distribution.

**Proof of Theorem 2** The result follows from the asymptotic normality of order statistics under simple random sampling without replacement [16]. The variance includes the finite population correction $f = (N-S)/N$, which accounts for the reduced variability due to sampling without replacement when $S$ is a non-negligible fraction of $N$.

**Proof of Theorem 4** Substituting Equations (8) and (11) into $M = ck$ gives

$$M = k\left[1 + \frac{z_A + z_B}{\sqrt{S}}\sqrt{\frac{1-\alpha}{\alpha}f}\right] = k + (z_A + z_B)\sqrt{\frac{k^2(1-\alpha)}{\alpha S}f}. \tag{15}$$

Letting $z = z_A + z_B$ and using $\alpha = k/N$, we obtain

$$\frac{k^2(1-\alpha)}{\alpha} = \frac{k^2(N-k)}{k} = k(N-k), \tag{16}$$

which yields

$$M = k + z\sqrt{\frac{k(N-k)}{S}}f. \tag{17}$$

This proves the claim.

**Proof of Theorem 5** Substituting $M = k + z\sqrt{k(N-k)/S}$ into Equation (13), differentiating with respect to $S$, and setting the derivative to zero gives

$$\frac{dC}{dS} = \tau_A - \frac{1}{2}\tau_B \log k \cdot z\sqrt{k(N-k)}\, S^{-3/2} = 0. \tag{18}$$

Solving for $S$ yields the optimizer stated in Equation (14). The second derivative is positive at this point, confirming that the solution is a minimum. In the sparse regime $k \ll N$, we have $k(N-k) \approx kN$, and therefore $S^\star \propto (kN)^{1/3}$.

## D Additional Algorithmic Details

**Scaling with $N$ and $k$.** The candidate buffer grows sublinearly with both problem size and sparsity level:

$$M - k \propto \sqrt{\frac{k(N-k)}{S}} = O\left(\sqrt{\frac{kN}{S}}\right).$$

Using the optimal sampling rate $S = \Theta((kN)^{1/3})$ yields

$$M - k = O\Big((kN)^{1/3}\Big),$$

which is asymptotically much smaller than $N$ in the practically relevant regime $k \ll N$. Thus, Prof-K replaces an exact selection problem over $N$ elements with one over a substantially smaller candidate set.

**Fallback as a Safety Mechanism** Rather than overprovisioning $S$ or $M$ to satisfy the target failure budget in pathological regimes (which, for very small $\alpha = k/N$, would require $S \propto 1/\alpha$, Prof-K reverts to exact top-$k$ whenever the probabilistic conditions are not met economically. Because such events are rare by design, their amortized cost is negligible in practice.

**Efficient GPU Implementation** All stages of Prof-K, i.e., sampling, threshold estimation and filtering, the main compute payload, are amenable to parallelization. In particular, the filtering scan can be efficiently implemented as a fused kernel in CUDA or in Triton, enabling different parts of the large tensor to be processed simultaneously, saving only a fraction of the data to a pre-allocated buffer.

**Why Prof-K Excels When $k$ is Small** When $k \ll N$, even a generous buffer multiplier $c = M/k$ corresponds to a modest absolute buffer size. For instance, with $N = 2^{29}$ and $k = 64$, the buffer may contain only a few thousand elements ($M \approx 5{,}000$), making the final exact top-$k$ extremely cheap relative to scanning half a billion elements.

**Distribution-Agnostic Guarantees** Unlike quantile-estimation methods that assume smooth or well-behaved distributions, Prof-K's guarantees stem from combinatorial properties of random sampling. The analysis depends only on ranks, not values, ensuring robustness against heavy-tailed, multimodal, or adversarial inputs.

**Graceful Degradation via Fallback** Rather than over-provisioning $S$ or $M$ to handle pathological corner cases, Prof-K simply falls back to exact top-$k$ when needed. If such events occur with probability $\varepsilon$, the amortized overhead remains $O(\varepsilon)$, which is negligible for typical choices like $\varepsilon = 10^{-3}$.